

CSN364- Container Technologies

Module-5

Topic 1- Kubernetes (K8s)

Kubernetes

- Kubernetes (K8s) is an open-source container orchestration platform.
- It is used to automate deployment, scaling, and management of containerized applications across multiple machines (nodes).
- Think of it as a super-powered Docker Swarm with more features for complex, production-grade environments.

Component	Role
Master/Control Plane	Manages the cluster, schedules workloads, keeps track of state
Node	Runs containers (pods) assigned by master
Pod	Smallest deployable unit, can contain one or more containers
Service	Exposes pods and handles load balancing
Deployment	Defines desired state, replicas, and updates

Why Do We Need Kubernetes if Docker Swarm Exists?

- Advanced Orchestration- Kubernetes supports auto-scaling, rolling updates, and self-healing, which Swarm handles in a limited way.
- Better for Large Clusters- Kubernetes can efficiently manage hundreds or thousands of nodes, whereas Swarm is best for small clusters.
- Stateful Application Support-Kubernetes handles databases and persistent storage seamlessly; Swarm mainly works for stateless apps.
- Multi-Cloud & Hybrid Deployments- Kubernetes works across different clouds and on-premises, Swarm is mostly single-cloud.
- Huge Ecosystem & Community Support- Kubernetes has a large ecosystem of monitoring, logging, networking, and CI/CD tools, making production setups easier.

“Docker Swarm is easy and good for learning or small projects. Kubernetes is needed when apps must scale, survive failures, run in multiple clouds, and stay highly available in production.”

Docker Swarm Vs Kubernetes

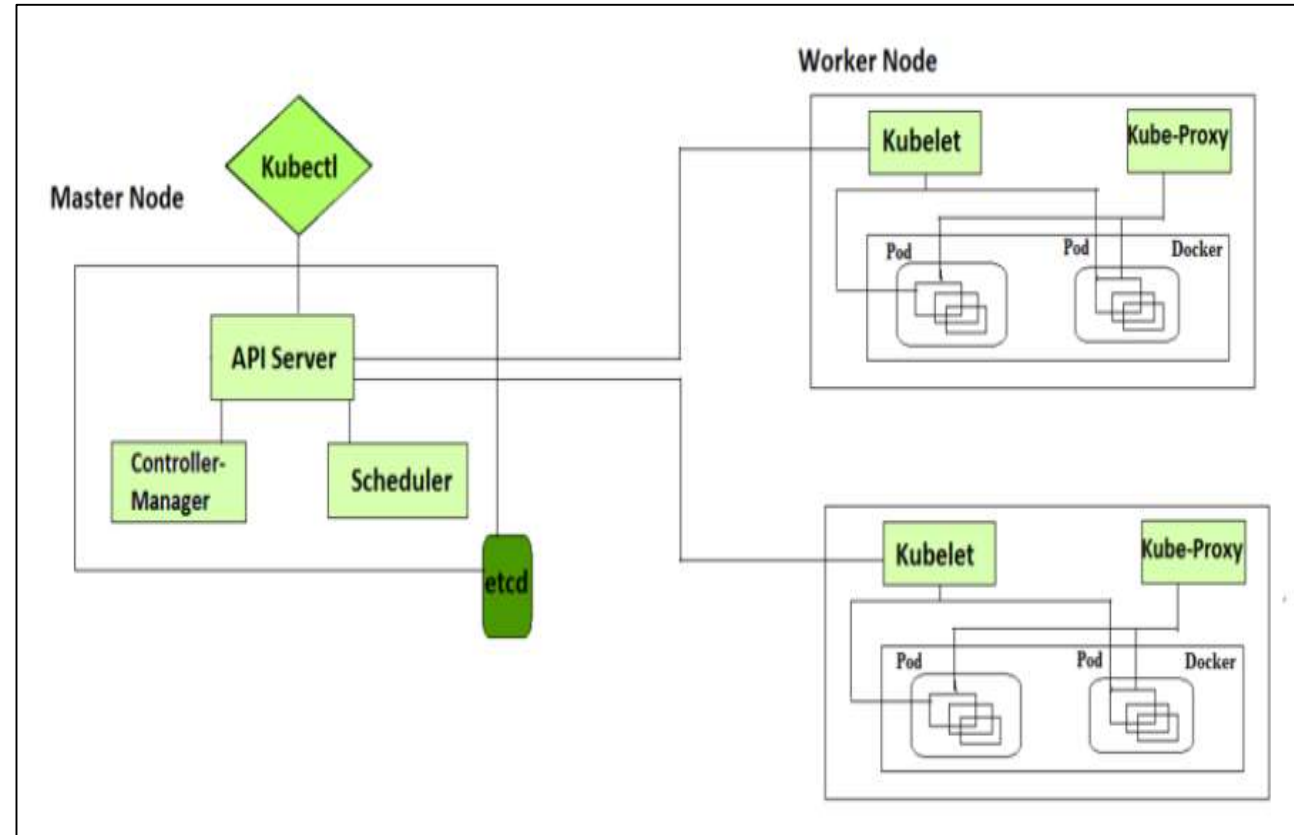
Feature	Docker Swarm	Kubernetes
Ease of Setup	Very simple, easy to get started	More complex, steeper learning curve
Scalability	Suitable for small to medium clusters	Designed for large-scale, production clusters
Architecture	Manager + Worker nodes, simpler	Master/Control Plane + Nodes, more components (Pods, Deployments, Services)
Auto-scaling	Limited	Advanced auto-scaling of pods and workloads
Self-healing	Basic (restarts failed containers)	Advanced (reschedules failed pods, maintains desired state)
Load Balancing	Basic internal and external load balancing	Advanced internal service load balancing and external ingress controllers
Persistent Storage	Limited support	Supports stateful workloads with persistent volumes
Multi-cloud/Hybrid Support	Minimal	Strong support for hybrid and multi-cloud deployments
Community & Ecosystem	Smaller, fewer tools	Large ecosystem, many tools, strong community
Rolling Updates & Rollbacks	Supported, but basic	Advanced rolling updates and automated rollbacks
Use Case	Simple projects, labs, small clusters	Production-grade apps, complex microservices, large clusters

Kubernetes Architecture

- Kubernetes comes with a client-server architecture. It consists of master and worker nodes.

Master Node (Control Plane)

- The master node manages the entire cluster and makes decisions like scheduling and responding to cluster events.
- **Kubectl**
 - This is the command-line tool you (the user) use to interact with the Kubernetes cluster.
 - Sends commands to the API Server.



Kubernetes Architecture

- **API Server**

- The main management point of Kubernetes.
- It exposes the Kubernetes API, receives requests (e.g., deploy an app), validates them, and processes them.
- Acts as the “front end” of the control plane.
- Communicates with all other components.

- **Scheduler**

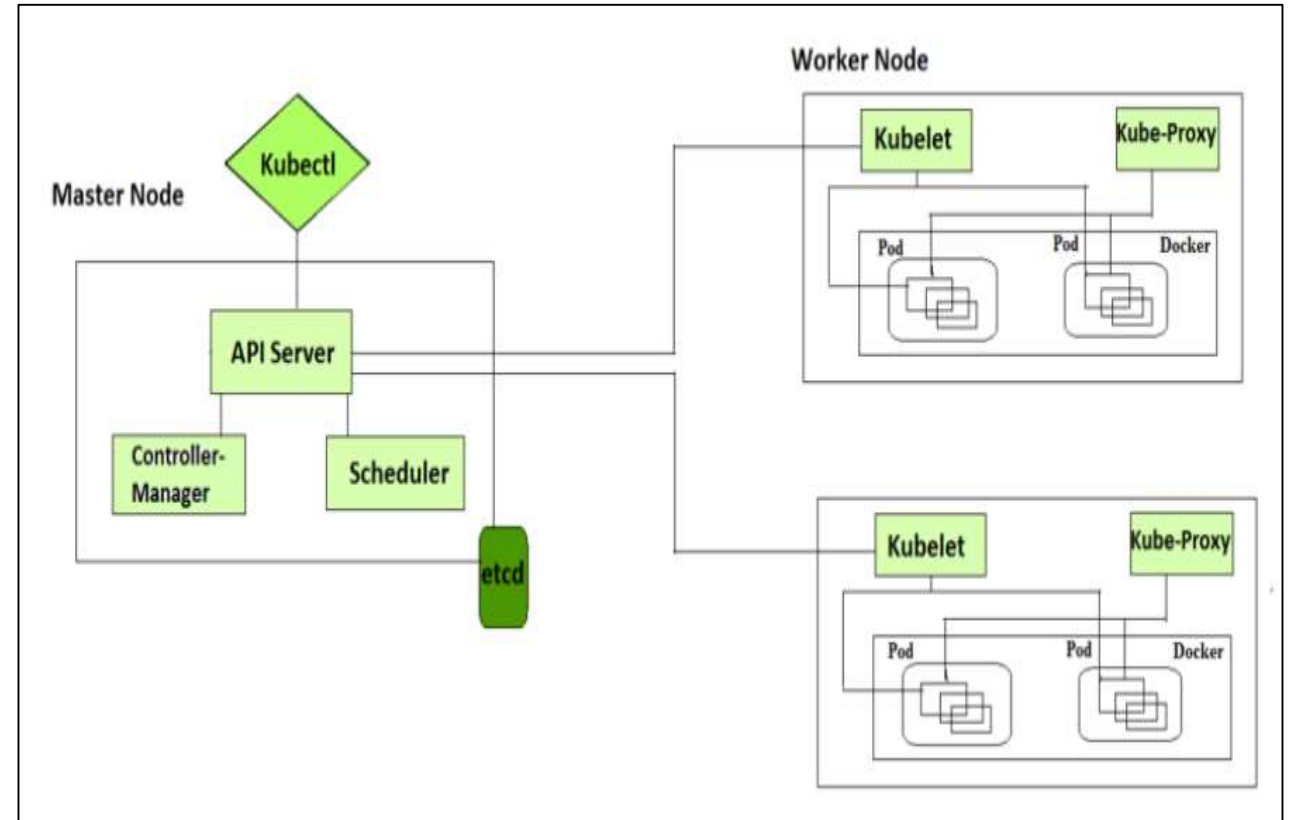
- Decides which worker node will run a newly created pod based on resource availability, policies, and other constraints.

- **Controller-Manager**

- Runs controller processes that handle routine tasks, such as replicating pods, handling node failures, and managing endpoints.

- **etcd**

- A key-value store that holds the cluster state and configuration data.
- It is the single source of truth for the cluster.



Kubernetes Architecture

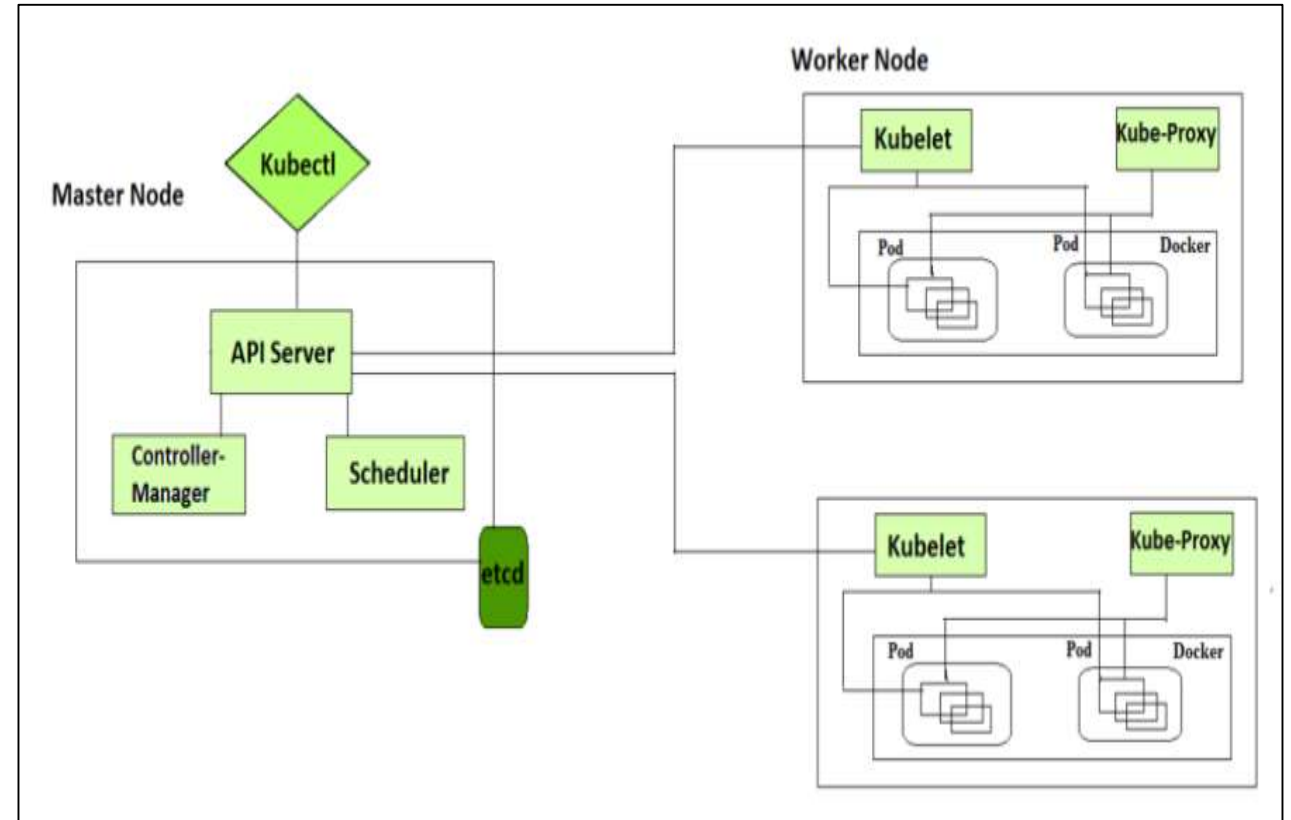
Worker Nodes- Worker nodes actually run your applications inside containers.

- **Kubelet**

- An agent running on each worker node.
- Receives instructions from the API Server and ensures containers are running as expected inside pods.
- Reports status back to the master.

- **Kube-Proxy**

- Handles networking on each node.
- Maintains network rules to allow communication to pods inside and outside the node.
- Manages service load balancing.



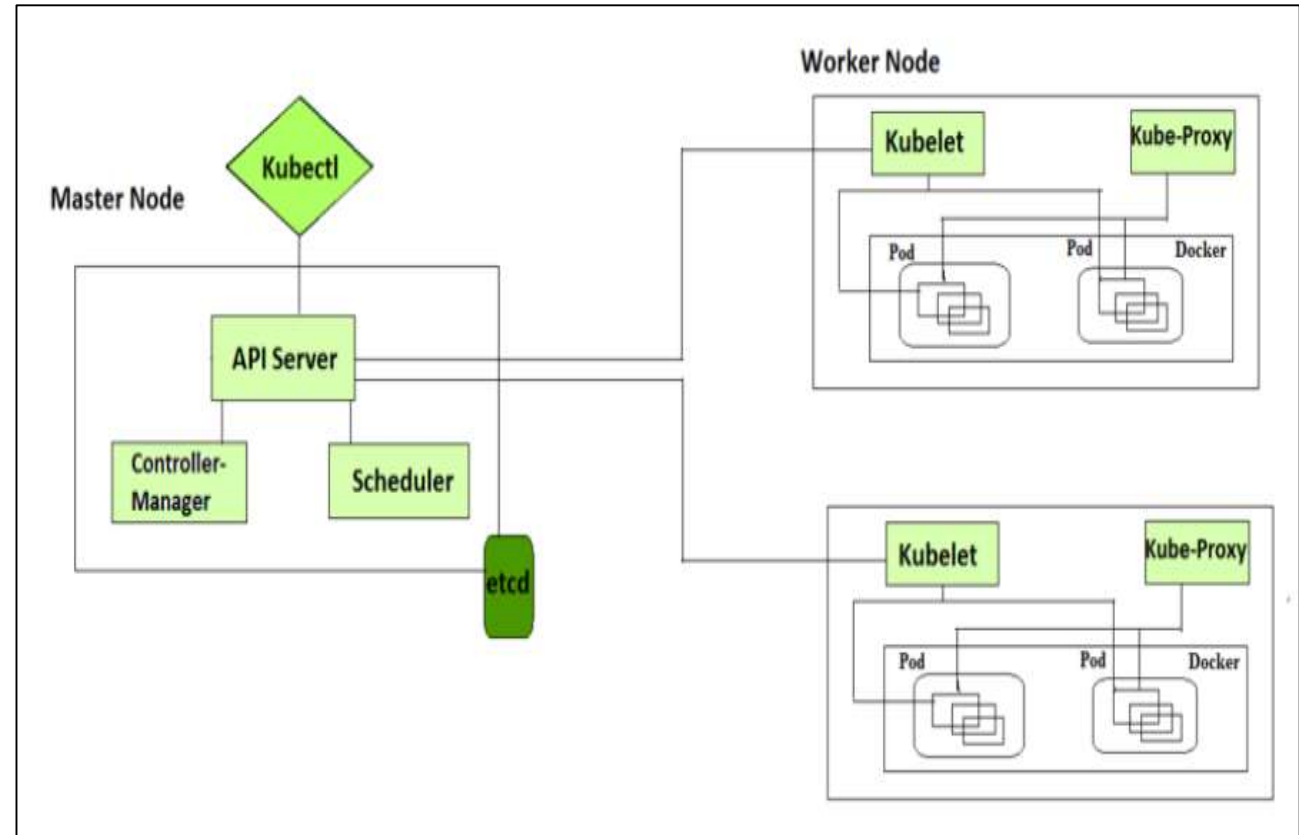
Kubernetes Architecture

- **Pods**

- The smallest deployable units in Kubernetes.
- A pod can contain one or more containers (usually one).
- Pods run on worker nodes.

- **Docker (or other container runtime)**

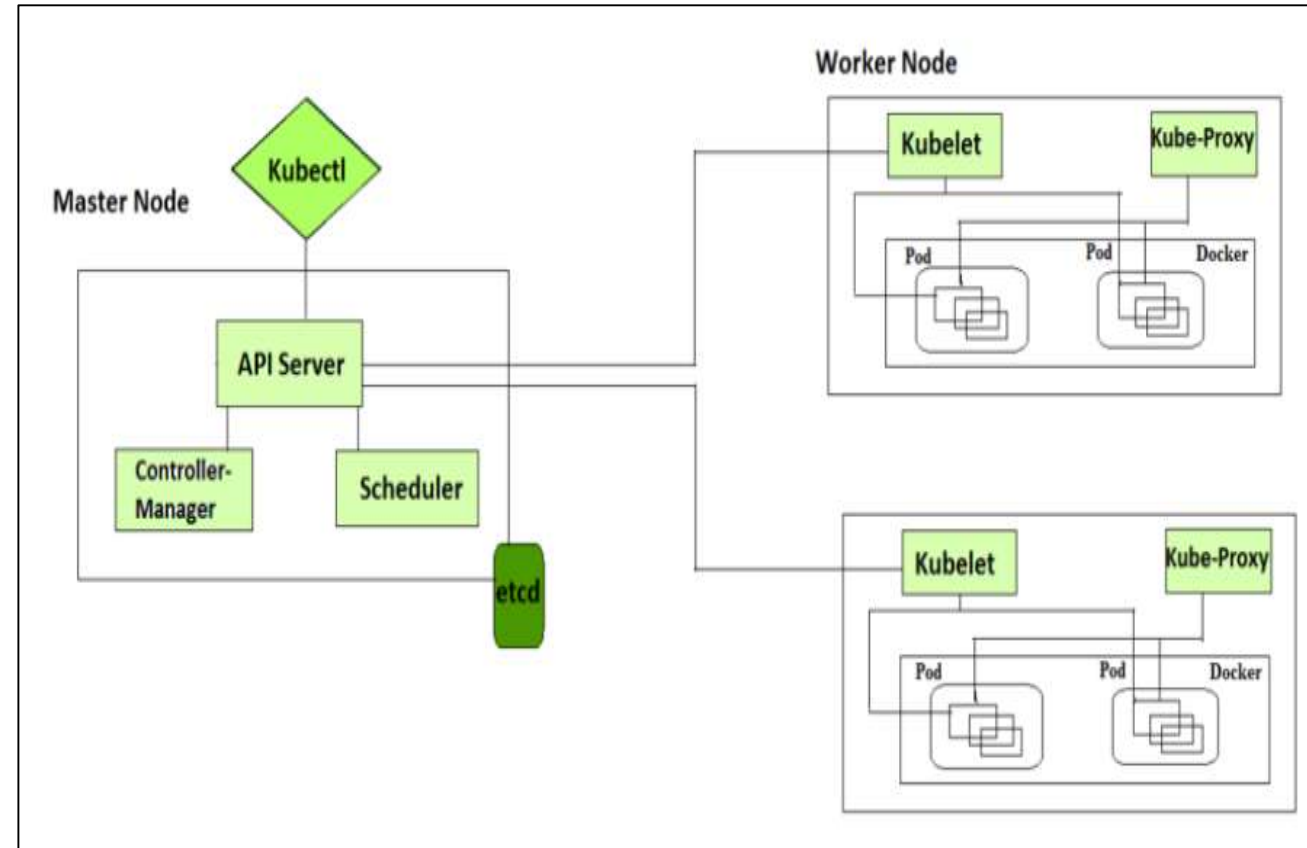
- The container runtime (Docker, containerd, etc.) that actually runs the containerized applications inside pods.



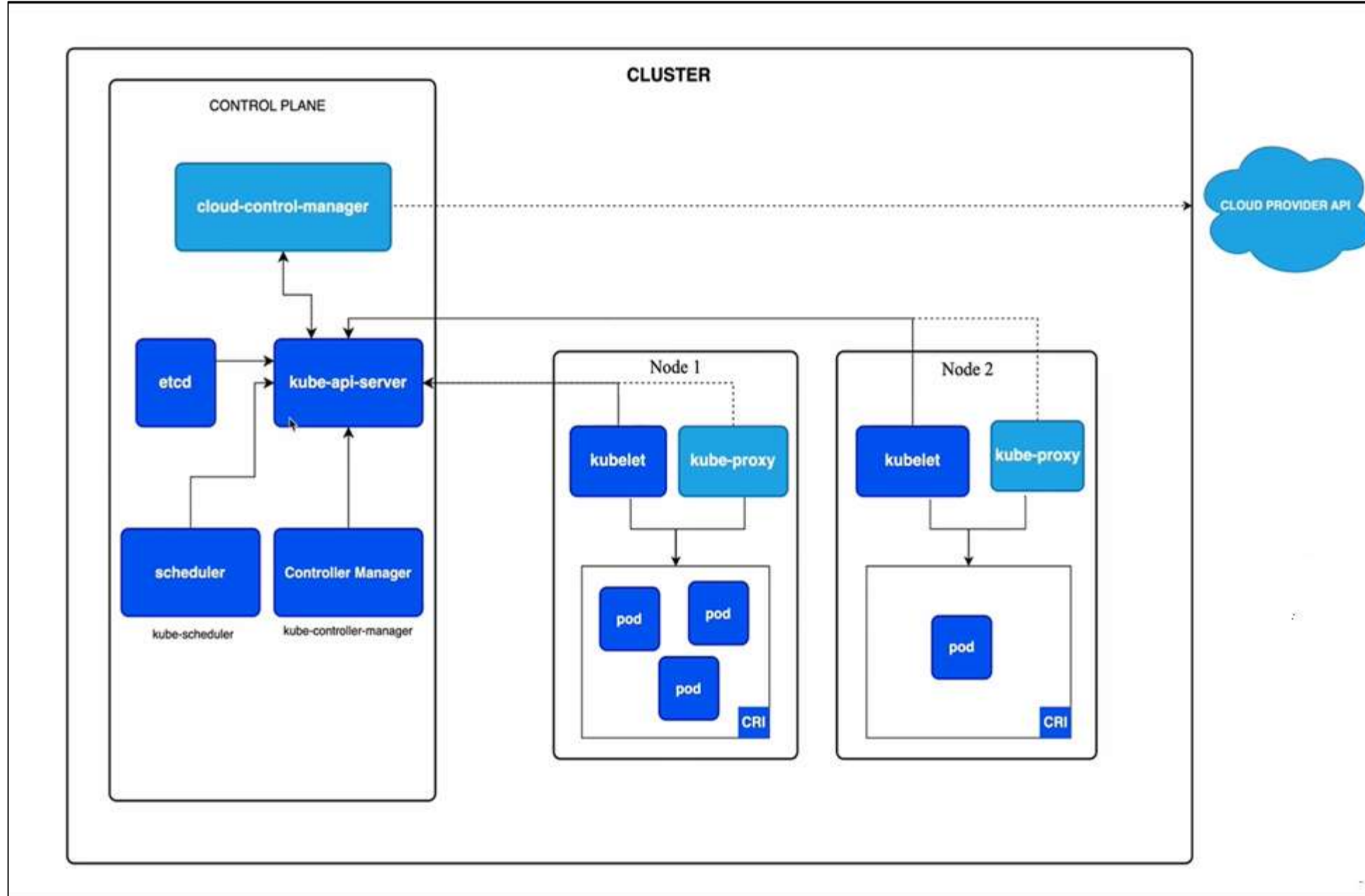
Kubernetes Architecture

How It Works Together

- When you run a command with **kubectl**, it talks to the **API Server** on the master.
- The **Scheduler** decides which worker node to run the pod on.
- The **Controller-Manager** keeps the system in the desired state (e.g., if a pod crashes, it creates a new one).
- The **Worker nodes** (via Kubelet) run the containers inside pods.
- **Kube-Proxy** manages network routing so services can communicate inside the cluster.
- All cluster data (states, configs) are stored in **etcd**.



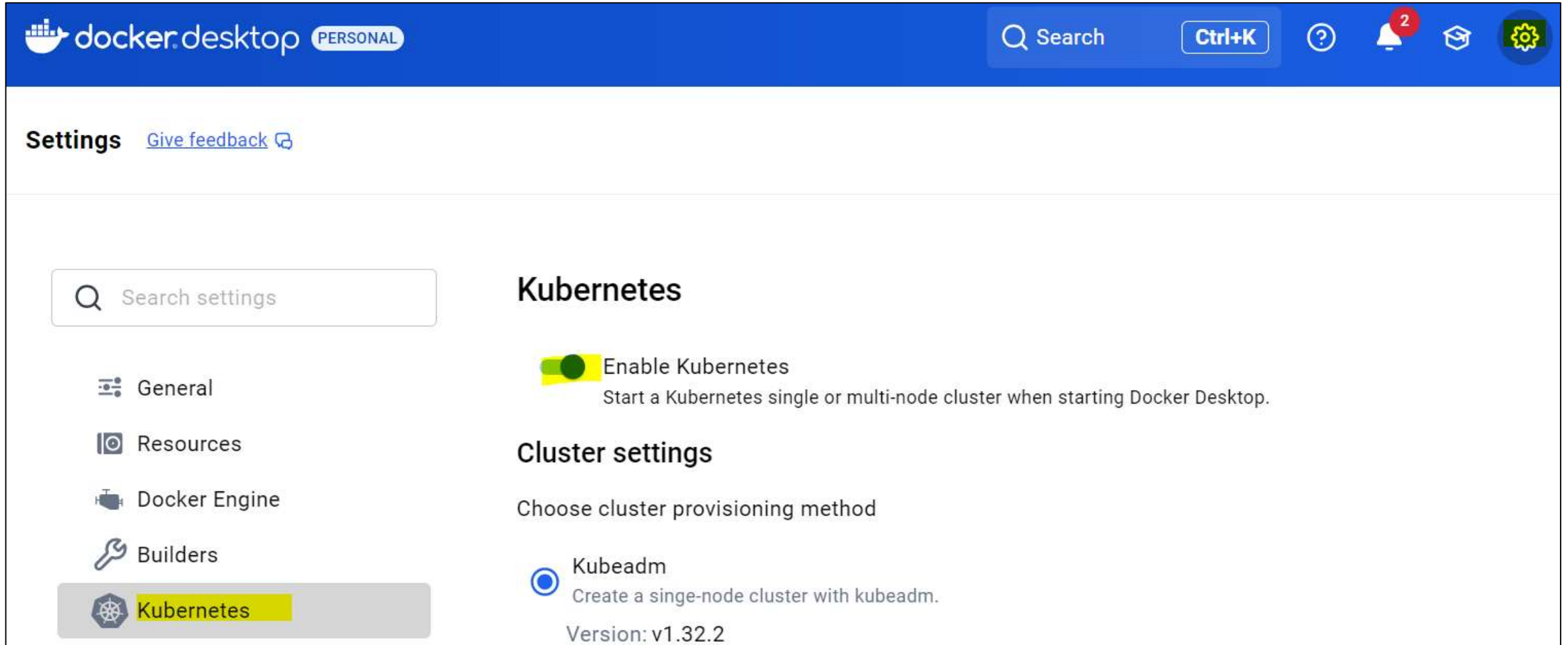
Kubernetes Architecture with Cloud load balancer



Kubernetes Lab Practice

To deploy a simple Python web application on Kubernetes using a Pod and a Service, and verify that it runs successfully inside the cluster.

Step-1 Enable Kubernetes on Docker desktop



Kubernetes Lab Practice

To deploy a simple Python web application on Kubernetes using a Pod and a Service, and verify that it runs successfully inside the cluster.

Step-2 Check Version (kubectl version --client)

```
PS C:\Users\DIT> kubectl version --client
Client Version: v1.32.2
Kustomize Version: v5.5.0
```

Step-3 Check no. of nodes (kubectl get nodes)

```
PS C:\Users\DIT> kubectl get nodes
NAME             STATUS    ROLES           AGE   VERSION
docker-desktop   Ready     control-plane   2m7s  v1.32.2
PS C:\Users\DIT>
```

Step-4 Create a folder kuber and inside make two files – app.py and Dockerfile

pyapp.py

```
from flask import Flask
app = Flask(__name__)
```

```
@app.route("/")
```

```
def home():
```

```
    return "Hello from Python inside Kubernetes!"
```

```
app.run(host="0.0.0.0", port=5000)
```

Kubernetes Lab Practice

To deploy a simple Python web application on Kubernetes using a Pod and a Service, and verify that it runs successfully inside the cluster.

Dockerfile

FROM python:3.10-slim

WORKDIR /app

COPY pyapp.py /app/pyapp.py

RUN pip install flask

EXPOSE 5000

CMD ["python", "pyapp.py"]

Step-5 Build Docker Image Locally (Make sure you are inside app folder Kuber)

docker build -t pyapp:1.0 .

Kubernetes Lab Practice

To deploy a simple Python web application on Kubernetes using a Pod and a Service, and verify that it runs successfully inside the cluster.

Step-6 Create Deployment YAML - deploy.yaml

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: pyapp-deployment
```

```
spec:
```

```
  replicas: 1
```

```
  selector:
```

```
    matchLabels:
```

```
      app: pyapp
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: pyapp
```

```
    spec:
```

```
      containers:
```

```
        - name: pyapp
```

```
          image: pyapp:1.0
```

```
          imagePullPolicy: Never
```

Kubernetes Lab Practice

To deploy a simple Python web application on Kubernetes using a Pod and a Service, and verify that it runs successfully inside the cluster.

Step-7 Create Service YAML - service.yaml

apiVersion: v1

kind: Service

metadata:

name: service

spec:

type: NodePort

selector:

app: pyapp

ports:

- port: 5000

targetPort: 5000

nodePort: 30005

Kubernetes Lab Practice

To deploy a simple Python web application on Kubernetes using a Pod and a Service, and verify that it runs successfully inside the cluster.

Step-8 Deploy to Kubernetes

kubectl apply -f deploy.yaml

kubectl apply -f service.yaml

Step-9 Verify Deployment

kubectl get pods

Kubectl logs <pod name>

Step-10 Check Deployment:

kubectl get deployments

Step-11 Check Service:

kubectl get services

Step-12 Get Node IP

kubectl get nodes -o wide

Step-13 Access the App in Browser

http://localhost:30005

Step-14 Delete Everything (Cleanup)

kubectl delete -f deploy.yaml

kubectl delete -f service.yaml

```
PS D:\DIT Notes\Containers Technology\Module-5\kube> kubectl apply -f deployment.yaml
deployment.apps/pyapp-deployment created
PS D:\DIT Notes\Containers Technology\Module-5\kube> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
pyapp-deployment-fb5cc74d-qt9qb     1/1     Running   0           28s
PS D:\DIT Notes\Containers Technology\Module-5\kube> kubectl get deployments
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
pyapp-deployment                    1/1     1             1           71s
PS D:\DIT Notes\Containers Technology\Module-5\kube> kubectl get services
NAME            TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
kubernetes     ClusterIP   10.96.0.1       <none>        443/TCP          28m
pyapp-service   NodePort    10.104.24.194   <none>        5000:30001/TCP   12m
```

